Tool installation

The course Docker image already contains the tools used in this practical. The instructions below install them separately on **Linux `x86-64`**. Choose **one method per tool**: Conda packages, or a source build. Source builds use Conda for runtime dependencies where useful. The Conda versions below match the practical. Source commands select release tags or pinned GitHub commits. The packaged Verkko release and the course's patched GitHub build are described separately.

Installation environment

Miniforge. If Conda is unavailable, download and install Miniforge, then initialize Conda in the current Bash terminal:

```
MINIFORGE_URL="https://github.com/conda-forge/miniforge/releases/latest/download"
curl -fL "$MINIFORGE_URL/Miniforge3-Linux-x86_64.sh" -o Miniforge3.sh
bash Miniforge3.sh -b -p "$HOME/miniforge3"
source "$HOME/miniforge3/etc/profile.d/conda.sh"
```

One environment. Use Python 3.11 and NetworkX 2.8 as in the course image. Keep all tools in this environment. Install the Python packages for Vizitig last.

```
conda create -n kmer-graph -c conda-forge python=3.11 pip setuptools wheel
conda activate kmer-graph
conda install -c conda-forge networkx=2.8
conda config --env --set channel_priority strict
INSTALL_PREFIX="$CONDA_PREFIX"
SOURCE_ROOT="$HOME/kmer-graph-sources"
mkdir -p "$SOURCE_ROOT" "$INSTALL_PREFIX/bin"
export PATH="$INSTALL_PREFIX/bin:$PATH"
```

Activate the environment in each new terminal. For source instructions, also set `INSTALL_PREFIX` and `SOURCE_ROOT` again. The source directory contains checkouts. The installation prefix is the active environment. Installing into this prefix needs no administrator privileges.

Miniforge installation.

Source build prerequisites

For source builds, install the compiler and development headers once. On Ubuntu or Debian, the following commands require administrator privileges:

```
sudo apt-get update
sudo apt-get install -y build-essential cmake git curl pkg-config
sudo apt-get install -y zlib1g-dev libbz2-dev liblzma-dev
```

Use GCC/G++ 10.2 or newer and CMake 3.16 or newer for the builds below. MEGAHIT and SPAdes need zlib. SPAdes also needs bzip2 development headers. The corresponding Conda package installations do not require these system build dependencies.

Rust for GGCAT and Verkko. Install a current stable Rust toolchain. GGCAT 2.2 uses Rust edition 2024, requiring Rust 1.85 or newer.

```
curl -fsSL https://sh.rustup.rs -o "$SOURCE_ROOT/rustup.sh"
sh "$SOURCE_ROOT/rustup.sh" -y --profile minimal --default-toolchain stable
source "$HOME/.cargo/env"
rustc --version
cargo --version
```

`source "$HOME/.cargo/env"` makes Rust available in the current terminal. Run it again in a new terminal if Rust is not already on `PATH`.

Installation checks. Each tool below has a version or help command. These check that the executable can start. They do not replace running an assembly or evaluation on your data.

Rust installation, SPAdes requirements, FastK source, GGCAT build documentation.

SPAdes

Conda. This installs SPAdes and its packaged dependencies.

```
conda install -c conda-forge -c bioconda spades=4.3.0
spades.py --version
```

From source. Requires Python, GCC/G++, CMake, zlib and bzip2 headers. The compilation script installs both the Python launchers and compiled tools into the active environment. Keep the generated supporting directories.

```
cd "$SOURCE_ROOT"
git clone --branch v4.3.0 --recursive https://github.com/ablab/spades.git
cd spades
PREFIX="$INSTALL_PREFIX" bash spades_compile.sh -j 4
spades.py --version
```

The command to run SPAdes remains `spades.py`. The compiled `spades-core` is a component of the pipeline rather than its usual entry point.

SPAdes installation documentation.

MEGAHIT

Conda.

```
conda install -c conda-forge -c bioconda megahit=1.2.9
megahit --version
```

From source. Requires GCC/G++, CMake and zlib headers. The recursive clone includes the required submodules. CMake's install step puts the launcher and its core binaries into the installation prefix.

```
cd "$SOURCE_ROOT"
git clone --branch v1.2.9 --recursive https://github.com/voutcn/megahit.git
cd megahit
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
-DCMAKE_INSTALL_PREFIX="$INSTALL_PREFIX"
cmake --build build --parallel 4
cmake --install build
megahit --version
```

Use `megahit` to run the pipeline. Keep `megahit_core` and the other installed binaries alongside it. Copying only the launcher is insufficient.

MEGAHIT installation documentation.

GGCAT

Conda.

```
conda install -c conda-forge -c bioconda ggcat=2.2.0
ggcat --version
```

From source. Requires the Rust toolchain above and a C/C++ build environment for native dependencies. `--locked` uses the dependency versions recorded in the release's Cargo lockfile. `--root` installs `ggcat` into the active environment's `bin` directory.

```
cd "$SOURCE_ROOT"
git clone --branch v2.2.0 https://github.com/algbio/ggcat.git
cargo install --path ggcat/crates/cmdline --locked --features kmer-counters --root
"$INSTALL_PREFIX"
ggcat --version
```

The practical uses the `ggcat build` subcommand to construct compacted de Bruijn graphs with links. The source command enables `kmer-counters` to include unitig abundance annotations, which Vizitig can import. Check for `km:f:` fields in the output FASTA headers. The course image's current GGCAT build omits them. Vizitig can also construct graphs directly from assembly FASTA files without running GGCAT separately.

Course image limitation. Direct `vizitig create` works at k = 31 and 63, but produced an empty graph at k = 127. For k = 127, build with the GGCAT executable and import, index and colour the graph with Vizitig. See the cheatsheet.

GGCAT source build documentation.

Flye

Conda.

```
conda install -c conda-forge -c bioconda flye=2.9.6
flye --version
```

From source. Requires Python, setuptools, GNU Make, a C++ compiler and zlib headers. Installing the checkout with pip compiles Flye's native components and installs the Python launcher into the active environment.

```
cd "$SOURCE_ROOT"
git clone --branch 2.9.6 --recursive https://github.com/mikolmogorov/Flye.git
python -m pip install --no-build-isolation ./Flye
flye --version
```

The source build includes Flye's own minimap2 and SAMtools components. The `flye` command starts the complete assembly pipeline.

Flye installation documentation.

hifiasm

Conda.

```
conda install -c conda-forge -c bioconda hifiasm=0.25.0
hifiasm --version
```

From source. Requires GNU Make, GCC/G++ and zlib headers. The build produces the `hifiasm` executable, which is copied into the active environment.

```
cd "$SOURCE_ROOT"
git clone --branch 0.25.0 https://github.com/chhylp123/hifiasm.git
make -C hifiasm -j 4
install -m 755 hifiasm/hifiasm "$INSTALL_PREFIX/bin/hifiasm"
hifiasm --version
```

This version includes `--ul` for combining HiFi and ultralong Nanopore reads. The installation method does not change the assembly command syntax.

hifiasm installation documentation.

Verkko

Conda release. The Bioconda package installs Verkko 2.3.2 and its runtime pipeline dependencies. Its recipe requires Snakemake 7, below version 8. This is a different installation from the course image's patched source build.

```
conda install -c conda-forge -c bioconda verkko=2.3.2 bc=1.07.1
verkko --version
snakemake --version
```

From source with Snakemake 9. The following uses the exact GitHub commit built for the course image. Verkko needs external programs as well as its own compiled modules. Install the runtime dependencies into the same environment:

```
conda install -c conda-forge -c bioconda snakemake-minimal=9.27.0 networkx=2.8
conda install -c conda-forge -c bioconda graphaligner mashmap winnowmap bc
conda install -c conda-forge -c bioconda samtools seqtk bwa minimap2 bamtools
conda install -c conda-forge -c bioconda parasail-python biopython pysam
```

Install the additional source build headers on Ubuntu/Debian:

```
sudo apt-get install -y libbamtools-dev libcurl4-openssl-dev libssl-dev libdeflate-dev
```

GCC/G++ and Rust are also required. Clone the repository, select the course commit, then initialize its submodules at that commit:

```
cd "$SOURCE_ROOT"
git clone https://github.com/marbl/verkko.git
cd verkko
git checkout cb69f969eebfa2b2a74003a120edabf2b084248f
git submodule update --init --recursive
```

Continue with the compatibility changes and compilation on the next page. Do not install the Conda Verkko package on top of this source build: its Snakemake constraint differs.

Verkko installation documentation, Bioconda runtime requirements.

Verkko source build: Snakemake 9 compatibility

Run these commands from the `verkko` checkout selected on the previous page. They apply the same two changes used in the course image: remove Snakemake's obsolete `--reason` flag, and give the haploid graph index rule its own output rather than declaring the shared empty file as an output.

```
sed -i '/--reason /d' src/verkko.sh
python - <<'PY'
from pathlib import Path
p = Path("src/Snakefiles/3-alignONT.sm")
text = p.read_text()
old = '"3-align/diploid.index"'
old += ''' if config['haploid'] == "False" else rules.emptyfile.output[0]'''
assert text.count(old) == 1
p.write_text(text.replace(old, '"3-align/diploid.index"'))
PY
```

Compile and install both the wrapper and supporting modules. The BamTools paths below are for Ubuntu/Debian `x86-64` with the system development package.

```
BAM_INC=/usr/include/bamtools
BAM_LIB=/usr/lib/x86_64-linux-gnu
make -C src -j 4 CC=gcc CXX=g++ BAMTOOLS_INCLUDE="$BAM_INC" BAMTOOLS_LIB="$BAM_LIB"
install -m 755 bin/verkko "$INSTALL_PREFIX/bin/verkko"
mkdir -p "$INSTALL_PREFIX/lib"
cp -a lib/verkko "$INSTALL_PREFIX/lib/"
verkko --version
snakemake --version
```

Verkko finds its modules relative to the installed wrapper. Keep the complete `lib/verkko` directory. The course image's patched Snakemake 9 build was run on the practical's local HiFi only and HiFi plus Nanopore examples.

Verkko requires accurate reads through `--hifi`. The practical's Nanopore like reads alone cannot start its pipeline through `--nano`.

Verkko input requirements.

SeqKit

Conda.

```
conda install -c conda-forge -c bioconda seqkit=2.14.0
seqkit version
```

From source. SeqKit is written in Go. Install Go, then compile the release checkout. This release's `go.mod` requests Go 1.25.14. Automatic toolchain selection downloads a suitable toolchain if the installed Go is older.

```
conda install -c conda-forge go
cd "$SOURCE_ROOT"
git clone --branch v2.14.0 https://github.com/shenwei356/seqkit.git
cd seqkit
GOTOOLCHAIN=auto go build -o "$INSTALL_PREFIX/bin/seqkit" ./seqkit
seqkit version
```

The result is the single `seqkit` executable. Its required Go modules are downloaded during the build. The Go compiler is not needed to run the result.

SeqKit source build documentation, release Go requirements.

QUAST

Conda.

```
conda install -c conda-forge -c bioconda quast=5.3.0
quast.py --version
```

From source. Requires Python and a compiler for the bundled alignment components. Install plotting dependencies, clone the release and install it with pip. QUAST's installation builds its core native tools.

```
conda install -c conda-forge numpy matplotlib
cd "$SOURCE_ROOT"
git clone --branch quast_5.3.0 https://github.com/ablab/quast.git
python -m pip install --no-build-isolation ./quast
quast.py --version
```

For this practical, the reference based comparison uses QUAST's core installation. Optional large genome and reference discovery components are not required for comparing the 500 kb assemblies with the supplied related reference.

QUAST installation documentation.

FastK

Conda. The course image uses FastK 1.2.

```
conda install -c conda-forge -c bioconda fastk=1.2
FastK
Histex
```

With no arguments, these commands print usage and return a nonzero status. This is expected. FastK's options are case sensitive.

From source. Requires a C compiler, GNU Make, Git and compression headers. HTSlib also needs libcurl and OpenSSL development headers on Ubuntu/Debian. Build and install the whole suite: MerquryFK calls its supporting utilities.

```
sudo apt-get install -y libcurl4-openssl-dev libssl-dev
cd "$SOURCE_ROOT"
git clone --recursive https://github.com/thegenemyers/FASTK.git
cd FASTK
git checkout 0e24fb45b71c4e14382ae1e1bc063bf66ea4e112
make -j 4
make install DEST_DIR="$INSTALL_PREFIX/bin"
FastK
Histex
```

For MerquryFK input, count with `-t1` to retain all kmers, including singletons. Keep the `.ktab`, `.hist` and hidden table files together.

FastK source and documentation.

MerquryFK

Conda. Installs MerquryFK 1.2 and FastK. Explicitly include the R plotting packages required by upstream, including those not listed by the Conda recipe.

```
conda install -c conda-forge -c bioconda merquryfk=1.2 fastk=1.2
conda install -c conda-forge r-argparse r-ggplot2 r-scales r-viridis r-cowplot
MerquryFK
```

The no argument command prints usage and returns a nonzero status. There is no `--help` or `--version` option. `-h` controls plot height.

From source. First install the complete FastK suite above. MerquryFK requires a C compiler and Make. R is needed at runtime to generate plots.

```
conda install -c conda-forge r-base r-argparse r-ggplot2 r-scales
conda install -c conda-forge r-viridis r-cowplot
cd "$SOURCE_ROOT"
git clone https://github.com/thegenemyers/MERQURY.FK.git
cd MERQURY.FK
git checkout 2f848f5f35c75d29838144d44b669c7f09839434
make -j 4
make install DEST_DIR="$INSTALL_PREFIX/bin"
MerquryFK
```

Keep FastK and the MerquryFK executables on `PATH`. No `MERQURY` variable is needed. Test a full evaluation and check the plots: a successful exit alone does not establish that the R dependencies work. The cheatsheet provides the counting and evaluation commands.

MerquryFK documentation and setup notes.

Bandage (BandageNG)

The course image contains **BandageNG 2026.9.1**. Its executable is `BandageNG`, including the capital letters. This is the maintained Bandage fork used below.

Conda. Install the package and its Qt runtime dependencies into the same active environment as the other tools:

```
conda install -c conda-forge -c bioconda bandage_ng=2026.9.1
QT_QPA_PLATFORM=offscreen BandageNG --version
```

From source. Requires a C++17 compiler, CMake 3.28 or newer, Qt 6 and its SVG development component. The recursive clone includes OGDF and `zlib-ng`. CMake downloads additional source dependencies, so configuration needs internet access. Install the build dependencies with Conda:

```
conda install -c conda-forge cmake=3.31 make cxx-compiler pkg-config qt6-main
conda install -c conda-forge libgl-devel libegl-devel
conda deactivate
conda activate kmer-graph
cd "$SOURCE_ROOT"
git clone --branch v2026.9.1 --recursive https://github.com/asl/BandageNG.git
cd BandageNG
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH="$CONDA_PREFIX"
cmake --build build --parallel 4
cmake --install build --prefix "$INSTALL_PREFIX"
QT_QPA_PLATFORM=offscreen BandageNG --version
```

CMake 3.31 satisfies this release's minimum and avoids changes introduced by CMake 4. Reactivating the environment enables the Conda compiler. Keep the Conda environment and its Qt libraries available when running the executable.

Interactive window. On a computer with a desktop display:

```
BandageNG load GRAPH.gfa --draw
```

A Docker terminal does not provide a desktop display by itself. For image export inside the course container, use the offscreen renderer:

```
QT_QPA_PLATFORM=offscreen BandageNG image GRAPH.gfa GRAPH.png --width 1600
```

`GRAPH.gfa` is the native assembler graph. `GRAPH.png` is the exported image.

BandageNG source documentation, Bioconda package.

Vizitig

Packaged installation with pip. The course uses the PyPI package. Install it into the active Conda environment, after the Conda tools above. The pinned NetworkDisk and NetworkX versions match the course image.

```
python -m pip install vizitig==1.2 vizibridge==0.6.2 networkdisk==1.1.9 networkx==2.8
vizitig --version
python -m pip check
```

From the source release. Download Vizitig's published source archive, then install that archive rather than its wheel. This uses the same 1.2 release without depending on the availability of GitLab access.

```
python -m pip download --no-deps --no-binary=:all: vizitig==1.2 -d "$SOURCE_ROOT"
python -m pip install vizibridge==0.6.2 networkdisk==1.1.9 networkx==2.8
python -m pip install "$SOURCE_ROOT/vizitig-1.2.tar.gz"
vizitig --version
python -m pip check
```

Vizitig itself is Python. The packaged `vizibridge` provides its Rust backed index. Pip uses a compatible wheel when available. To compile that dependency as well, use the Rust toolchain above:

```
python -m pip install --force-reinstall --no-binary=vizibridge vizibridge==0.6.2
```

Browser interface. The server listens on port 4242. Start it locally, then open the displayed URL in your browser:

```
export VIZITIG_DIR="$PWD"
vizitig run --no-launch-browser
```

The export uses the current writable directory as Vizitig's store. Use the same directory when importing graphs, indexing, colouring and starting the server.

Open http://localhost:4242. If using the course Docker image, publish that port when starting the container:

```
docker run --rm -it -p 127.0.0.1:4242:4242 npmalfoy/scalable:2026 bash
export VIZITIG_DIR="$PWD"
vizitig run --hostname 0.0.0.0 --no-launch-browser
```

To access input files in your current host directory and retain outputs after exit, use this alternative Docker command from that directory. Then set `VIZITIG_DIR` and start Vizitig inside the container as above.

```
docker run --rm -it --user "$(id -u):$(id -g)" -v "$PWD:$PWD" -w "$PWD" -p
127.0.0.1:4242:4242 npmalfoy/scalable:2026 bash
```

Vizitig repository, published package and source release.